arXiv:2407.08250v3 [cs.LG] 18 Oct 2026

Gradient Boosting Reinforcement Learning

Benjamin Fuhrer [1] **Chen Tessler** [2] **Gal Dalal** [2]

Abstract

We present Gradient Boosting Reinforcement Learning (GBRL), a framework that adapts the strengths of gradient boosting trees (GBT) to reinforcement learning (RL) tasks. While neural networks (NNs) have become the de facto choice for RL, they face significant challenges with structured and categorical features and tend to generalize poorly to out-of-distribution samples. These are challenges for which GBTs have traditionally excelled in supervised learning. However, GBT's application in RL has been limited. The design of traditional GBT libraries is optimized for static datasets with fixed labels, making them incompatible with RL's dynamic nature, where both state distributions and reward signals evolve during training. GBRL overcomes this limitation by continuously interleaving tree construction with environment interaction. Through extensive experiments, we demonstrate that GBRL outperforms NNs in domains with structured observations and categorical features while maintaining competitive performance on standard continuous control benchmarks. Like its supervised learning counterpart, GBRL demonstrates superior robustness to out-of-distribution samples and better handles irregular state-action relationships.

1. Introduction

Many real-world decision-making tasks involve structured observations, where data can be organized in a tabular format and follow predefined organizational patterns. Unlike unstructured data (such as images or raw sensor data), structured observations often include heterogeneous features, both numerical and categorical. These features carry direct semantic meaning that can be used for prediction without complex feature extraction. Such structured observations closely resemble tabular datasets commonly encountered in supervised learning. Notable examples where structured observations are crucial include recommendation systems, healthcare diagnostics, digital advertising, fraud detection, weather prediction, and financial trading.

In these domains, a key challenge lies in handling structured observations, where the importance of different components varies dynamically with the agent's state and task. Neural network (NN)-based solutions struggle with such data, requiring sophisticated architectures and preprocessing techniques to capture structural relationships (Zabërgja et al., 2024; Kadra et al., 2021; Arik & Pfister, 2021; Gorishniy et al., 2021; Hollmann et al., 2023). This often results in complex models that sacrifice sample efficiency and generalization capability.

Gradient boosting trees (GBT) offer a promising alternative through their natural handling of structured data. Their success in supervised learning stems from an iterative ensemble-building process, in which each decision tree refines predictions by partitioning the input space along individual features. This approach has made GBT frameworks such as XGBoost (Chen & Guestrin, 2016), LightGBM (Ke et al., 2017), and CatBoost (Prokhorenkova et al., 2018) integral in domains such as finance (Tian et al., 2020), healthcare (Wassan et al., 2022; Ma et al., 2022; Seto et al., 2022), and competitive data science (Chen, 2023).

This approach creates an ensemble of piecewise constant functions that excel at capturing non-smooth patterns and abrupt transitions in data (Jeffares et al., 2024; Beyazit et al., 2023; Grinsztajn et al., 2022). These properties are particularly valuable in RL, where state-action relationships often exhibit sudden changes, for instance, when transitioning between different phases of a task or when certain state variables cross critical thresholds. Such discontinuous relationships are common in structured RL tasks, from inventory management where actions change discretely with stock levels, to game scenarios where strategic decisions shift abruptly based on state conditions.

Despite these potential benefits, adapting GBT to RL presents significant challenges. Traditional GBT frameworks are designed for static datasets with stable feature distributions and predefined loss functions, where the focus

[1]NVIDIA, Tel-Aviv, Israel [2]NVIDIA Research, Tel-Aviv, Israel. Correspondence to: Benjamin Fuhrer <bfuhrer@nvidia.com>, Chen Tessler <ctessler@nvidia.com>, Gal Dalal <gdalal@nvidia.com>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

is on optimizing batch training efficiency. This contrasts sharply with RL's dynamic nature, where state distributions evolve during training, data is collected iteratively, and rewards are often delayed. While traditional GBT libraries can technically be adapted to RL using custom loss functions and incremental learning, our experiments with CatBoost and XGBoost (Section 4.3) show that this approach fails to scale due to their supervised learning-oriented design.

We address these challenges through Gradient Boosting Reinforcement Learning (GBRL), a framework that bridges the gap between GBT's strengths and RL's unique requirements. Our contributions are:

1. New framework for GBT in RL: We establish GBT as a viable function approximator for RL by devising popular algorithms—PPO, A2C, and AWR—with GBT backends. In popular environments, our framework demonstrates competitive performance against NNs while showing particular strengths on structured tasks.

2. Improved robustness: We demonstrate that GBT's inherent strengths translate effectively to RL tasks. Through extensive dedicated experiments, we show superior robustness to out-of-distribution scenarios, resilience to spurious correlations, and better handling of state-space perturbations – critical advantages over NNs for real-world applications.

3. Practical implementation: We provide a CUDA-accelerated (NVIDIA, 2025) implementation that seamlessly integrates with existing RL libraries like Stable-baselines3 (Raffin et al., 2021)[1] [2] Our shared Actor-Critic design achieves state-of-the-art performance while significantly reducing both memory and computational requirements on modern hardware.

2. Related Work

GBT Beyond Supervised Learning. Recent advances have extended the capabilities of GBT beyond traditional regression and classification. In ranking problems, GBT has been used to directly optimize ranking metrics (Lyzhin et al., 2023), as demonstrated by frameworks such as StochasticRank (Ustimenko & Prokhorenkova, 2020). Furthermore, GBT offers probabilistic predictions through frameworks such as NGBoost (Duan et al., 2020), allowing uncertainty quantification (Malinin et al., 2021). The connection between GBT and Gaussian processes (Ustimenko et al., 2023; Sigrist, 2022) offers further possibilities for uncertainty-aware modeling. Recently, Ivanov & Prokhorenkova (2021) modeled graph-structured data by combining GBT with graph NNs.

Despite their versatility, applying GBT in RL remains a relatively less explored area. Several works have employed GBT as a function approximator within off-policy RL methods, including its use in Q-learning (Abel et al., 2016) and in bandit settings to learn inverse propensity scores (London et al., 2023). Recently, Brukhim et al. (2022) proposed a boosting framework for RL where a base policy class is incrementally enhanced using linear combinations and nonlinear transformations, forming a 2-layer NN. However, these previous works have not demonstrated scalability and effectiveness in complex, high-dimensional RL environments that require extensive interactions. In this work, we show how to adapt the framework of GBT to successfully solve large-scale RL problems for the first time.

Tabular Data. Previous work in RL has focused predominantly on the use of NNs due to their ability to capture complex patterns in high-dimensional data. Techniques such as Q-learning and Actor-Critic methods have advanced significantly, demonstrating success in tasks involving raw sensory inputs like images, text, and audio. However, NNs that perform well on tabular data typically have very specialized architectures (Katzir et al., 2021; Somepalli et al., 2021; Gorishniy et al., 2021; Arik & Pfister, 2021) and are different from the standard multi-layer perceptrons that are often used in RL tasks and algorithms (Ota et al., 2024). Even with specialized architectures, GBT often performs equally or better on tabular datasets (Borisov et al., 2021; McElfresh et al., 2023; Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022). GBT's success can be attributed to a key difference in inductive biases compared to NNs. Jeffares et al. (2024) have shown that the kernel representations induced by GBTs are bounded and often behave more predictably on irregular or out-of-distribution inputs. In contrast, NN-tangent kernels can be unbounded and vary significantly on test points far from the training distribution.

Policy Optimization through Functional Gradient Ascent. Kersting & Driessens (2008) introduced Non-Parametric Policy Gradients (NPPG). By combining the policy gradient theorem (Sutton et al., 1999) with functional gradient ascent, NPPG directly optimizes the policy. Although NPPG sets the foundation for combining GBT with RL, it is limited to discrete action spaces and policies represented by the Gibbs distribution. GBRL extends the ideas laid in NPPG to general Actor-Critic algorithms and generalizes to any policy representation and objective.

3. RL Preliminaries

A fully observable infinite-horizon MDP is characterized by the tuple $(\mathcal{S}, \mathcal{A}, P, \mathcal{R})$. At each step, the agent observes

[1]The GBRL core library is available at https://github.com/NVlabs/gbrl.

[2]Actor-Critic implementations integrated within Stable-baselines3, are available at https://github.com/NVlabs/gbrl_sb3.

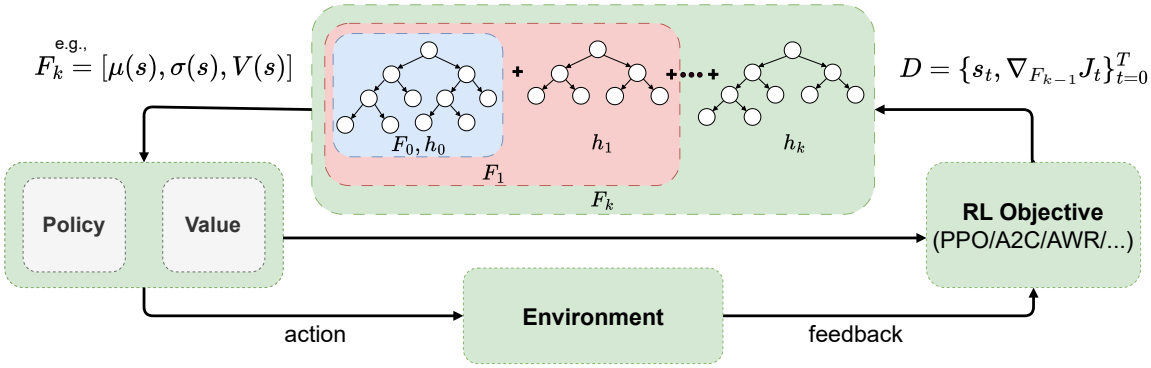


Figure 1. **The GBRL framework.** The actor's policy and critic's value function are parameterized by the tree ensemble F_k. For example, $F_k(\mathrm{s}) = [\mu(\mathrm{s}), \sigma(\mathrm{s}), V(\mathrm{s})]$ for a Gaussian policy. Starting from F_0, at each training iteration, k, GBRL collects a rollout and computes the gradient $\nabla_{F_{k-1}} J(\pi_{F_{k-1}})$ with respect to the current ensemble. This gradient is then used to fit the next tree. Adding the tree to the ensemble updates it to $F_k(\mathrm{s}) = F_{k-1}(\mathrm{s}) + \epsilon h_k(\mathrm{s})$, where ϵ is the learning rate.

a state $\mathrm{s} \in \mathcal{S}$ and samples an action $\mathrm{a} \in \mathcal{A}$ from its policy $\pi(\mathrm{s}, \mathrm{a})$. The action causes the system to transition to a new state s' based on the transition probabilities $P(\mathrm{s}' \,|\, \mathrm{s}, \mathrm{a})$, and produce a reward $\mathrm{r} \sim \mathcal{R}(\mathrm{s}, \mathrm{a})$. The agent's objective is to maximize the expected discounted reward $J(\pi) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t \mathrm{r}_t]$, with a discount factor $\gamma \in [0, 1)$.

The state-action value function $Q_\pi(\mathrm{s}, \mathrm{a}) := \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t \mathrm{r}_t \,|\, \mathrm{s}_0 = \mathrm{s}, \mathrm{a}_0 = \mathrm{a}]$ estimates the expected returns of performing action a in state s and then acting according to π. Additionally, the value function $V_\pi(\mathrm{s}) := \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t \mathrm{r}_t \,|\, \mathrm{s}_0 = \mathrm{s}]$, predicts the expected return starting from state s and acting according to π. Finally, the advantage function $A_\pi(\mathrm{s}, \mathrm{a}) := Q_\pi(\mathrm{s}, \mathrm{a}) - V_\pi(\mathrm{s})$, indicates the expected relative benefit of performing action a over acting according to π.

Actor-Critic methods are a common approach in RL. They simultaneously learn a policy and a value function to improve the efficiency and stability of training. For conciseness, in the body of the paper we focus on Proximal Policy Optimization (PPO) (Schulman et al., 2017) which jointly trains an actor and critic. In the supplementary material, we also showcase the implementation and results for A2C (Mnih et al., 2016) and AWR (Peng et al., 2019).

4. Gradient Boosting Reinforcement Learning

Deep RL with NNs faces significant challenges when dealing with structured observations containing both numerical and categorical features. Similarly to tabular datasets in supervised learning, these observations consist of heterogeneous features representing fundamentally different types of information, such as numerical measurements alongside categorical attributes describing system states or task conditions. These features create highly irregular patterns, as they may independently affect the environment based on their context, scale, or statistical properties. NNs struggle to model such patterns effectively (Beyazit et al., 2023; Jeffares et al., 2024), and their performance heavily depends on preprocessing methods (Hancock & Khoshgoftaar, 2020; Gorishniy et al., 2022). In contrast, GBT naturally handles irregular patterns and structured feature spaces efficiently. To leverage this strength, we present GBRL, an adaptation of GBT to RL tasks with such representations that frequently arise in real-world problems.

4.1. GBT as Functional Gradient Descent

GBT (Friedman, 2001) combines decision tree ensembles with functional gradient descent to learn complex nonlinear functions (Mason et al., 1999). Given a loss function L and a dataset, $D = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N}$, where $\mathbf{x}_i \in \mathbb{R}^M$ and $\mathbf{y}_i \in \mathbb{R}^D$, GBT aims to learn a function F, mapping inputs $\mathbf{x}$ to outputs $\mathbf{y}$. Through functional gradient descent, GBT minimizes the expected loss $\mathbb{E}_{\mathbf{x},\mathbf{y}}[L(\mathbf{y}, F(\mathbf{x}))]$ with respect to the learned function F. Unlike parametric gradient descent, which updates parameters, functional gradient descent updates a learned function directly.

Starting with an initial function F_0. At each iteration k, GBT refines F_{k-1}, by taking a step in the negative direction of the gradient $g_k := \nabla_{F_{k-1}} L(\mathbf{y}, F_{k-1}(\mathbf{x}))$. The negative gradient is projected onto the space of possible trees $h \in \mathcal{H}$ by constructing a new binary decision tree that minimizes the following objective:

$$h_k = \arg\min_{h \in \mathcal{H}} \| - g_k - h(\mathbf{x}) \|_2^2 \,. \tag{1}$$

Through this process, GBT minimizes the expected loss, resulting in the additive model:

$$F_K(\mathbf{x}) = F_0 + \sum_{k=1}^{K} \epsilon h_k(\mathbf{x}) \,, \tag{2}$$

Library	Incremental Learning (sec)	Standard Learning (sec)
CatBoost	36.17 ± 2.77	1.15 ± 0.03
XGBoost	67.11 ± 0.70	2.26 ± 3.70
GBRL	$\mathbf{5.47 \pm 1.19}$	N/A

Table 1. **Incremental Learning Speed in GBT Libraries.** Training times (seconds) for 1000 boosting iterations on random batches of 128 samples with 20 input features and 3 targets. While CatBoost and XGBoost train efficiently in standard mode using the same batch for all iterations, their training time increases significantly in incremental learning, where each batch receives only one boosting iteration. GBRL, designed explicitly for incremental updates, remains much faster.

where, ϵ is the learning rate controlling the step size in each iteration.

While standard GBT frameworks excel at handling structured data in supervised tasks, they require a tailored solution for RL's iterative, dynamic, and multi-objective nature.

4.2. The GBRL Framework

In the GBRL framework (Figure 1) we adapt GBTs to handle the challenges of RL. To do so we reinterpret $F_K(\mathrm{s})$ as the parameterization of the policy and value function, where the observed state $\mathrm{s} \in \mathcal{S}$ is the input to the ensemble.

Specifically, at each iteration k, we compute the policy gradient (Sutton et al., 1999). As opposed to the original formulation, where the gradient is calculated with respect to parameter space, here we calculate it in function space. Consequently, we express the policy gradient as:

$$\nabla_{F_{k-1}} J(\pi_{F_{k-1}}) = \mathbb{E}_{\pi_{F_{k-1}}}[\nabla_{F_{k-1}} \log \pi_{F_{k-1}}(\mathrm{a} \,|\, \mathrm{s}) A(\mathrm{s}, \mathrm{a})], \tag{3}$$

where F_{k-1} is the current ensemble-based parametrization.

Then, the policy gradient is projected onto a new tree and added to the ensemble:

$$h_k = \arg\min_{h \in \mathcal{H}} \|\nabla_{F_{k-1}} J(\pi_{F_{k-1}}) - h(\mathrm{s})\|_2^2 . \tag{4}$$

The result is an application of GBT as a functional gradient optimizer, updating both actor and critic incrementally:

$$F_K(\mathrm{s}) = F_0 + \sum_{k=1}^{K} \epsilon h_k(\mathrm{s}) \approx F_0 + \sum_{k=1}^{K} \epsilon \nabla_{F_{k-1}} J(\pi_{F_{k-1}}). \tag{5}$$

Gradient-based Approach. As seen in Equation (5), GBRL is an online learning gradient-based optimizer. In each step, a new tree is constructed to minimize the sampled loss. This learning scheme is identical to the common policy gradient methods (Mnih et al., 2016). By leveraging optimization frameworks for gradient computation, such as PyTorch (Paszke et al., 2019), GBRL can be integrated with most Actor-Critic algorithms and implemented within existing RL libraries. In contrast, traditional GBT frameworks are designed for offline, static datasets and require significant customizations and workarounds to function in RL.

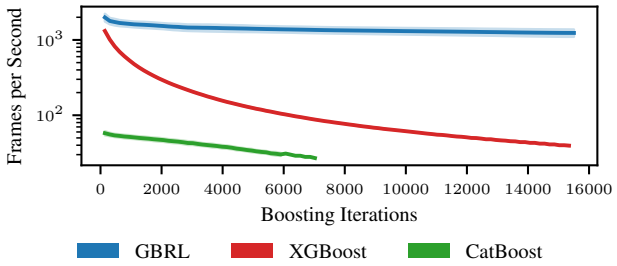


Figure 2. **GBT library comparison, Cartpole.** CatBoost and XGBoost are intractable in RL. CatBoost's lack of GPU support for custom losses leads to low FPS and early termination.

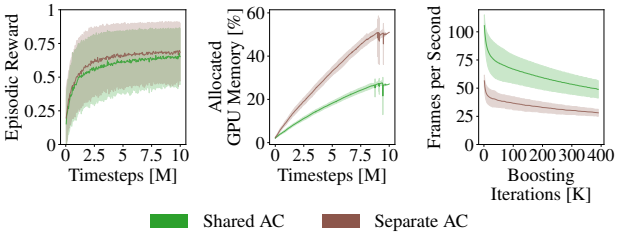


Figure 3. **Shared Actor-Critic, MiniGrid.** The shared tree structure significantly increases efficiency, without impacting the score. Aggregated results over three tasks are shown here, full per-task curves are available in Appendix (Figure 20)

Shared Actor-Critic Architecture. Many RL algorithms operate with a shared Actor-Critic structure. This structure helps avoid overfitting and improves overall convergence speeds (Henderson et al., 2018; Andrychowicz et al., 2020). We adopt this approach in GBRL such that each leaf produces two predictions. GBRL predicts both the policy (distribution over actions) and the value estimate. The internal structure of the tree is shared, providing a single feature representation for both objectives and significantly reduces memory and computational bottlenecks. Similar to the standard practice, we support and use separate learning rates for the policy and value. This approach enables solving two distinct objectives within a single shared structure.

4.3. Comparison to Traditional GBT Libraries

Before diving into the comparisons with NNs, Section 5, we first evaluated our design choices against traditional GBT libraries. While these libraries can be adapted for RL using custom loss functions and incremental learning, experiments with CatBoost and XGBoost reveal scalability

issues inherent to their supervised learning design. Table 1 shows these frameworks are significantly slower than GBRL, when used for incremental learning. Incremental learning is a fundamental property of RL, as the policy is continually changing. Moreover, when training on CartPole (Figure 2), larger ensemble sizes lead to significant drops in training throughput. In contrast, our shared Actor-Critic architecture halves memory usage and doubles throughput without compromising policy quality (Figure 3). Overall, GBRL offers a purpose-built solution that maintains GBT's advantages while efficiently handling RL-specific needs.

5. Experiments and Results

Our experiments aim to answer two core questions:

1. GBT as an RL function approximator: Can GBT-based AC algorithms effectively solve complex high-dimensional RL tasks? And how do they compare with NNs?

2. Advantages of GBT: Building on the GBT's success in irregular and tabular data, does its inductive bias offer similar robustness benefits in RL – specifically for out-of-distribution states, noisy inputs, and spurious correlations?

For our experiments, we implemented a GBT-based version of PPO within Stable Baselines3 (Raffin et al., 2021). Where available, we use standard hyperparameters, environment-specific, and normalization wrappers according to RL Baselines3 Zoo (Raffin, 2020); otherwise, we optimize the hyperparameters for specific environments. For each experiment, we report the aggregated performance across five random seeds. Our training setup consists of a single NVIDIA V100 GPU. We refer the reader to the supplementary material for additional technical details, such as hyperparameters, implementation, and environment details (Section C), full results (Section D) and full learning curves (Section E) for all evaluated algorithms.

5.1. Standard Environments

Experimental Setup. We evaluated GBRL against NNs across three categories of RL benchmarks. First, we tested **classic RL** tasks using Classic-Control and Box2D environments from Gymnasium (Towers et al., 2024). These provided a baseline for comparison on standard benchmarks. We then evaluated both methods on more complex environments with high-dimensional vectorized representations: the **Football** domain (Kurach et al., 2020), where we employed the built-in 'Checkpoints' shaped reward, and the **Atari RAM** domain (Bellemare et al., 2013). Finally, we assessed performance on **categorical environments**, specifically targeting the MiniGrid domain (Chevalier-Boisvert et al., 2023) – a setting where GBTs have traditionally excelled in supervised learning. MiniGrid provides an ideal test environment through its 2D grid-world structure featuring goal-oriented tasks with discrete object interactions.

Results. We report the cumulative non-discounted reward, averaged across the last 100 episodes in Figure 4. For a simple visual comparison between GBRL and NNs, we report the normalized score: $r_{norm} = \frac{r_{GBRL} - r_{NN}}{r_{max\{NN,GBRL\}} - r_{min\{NN,GBRL\}}}$.

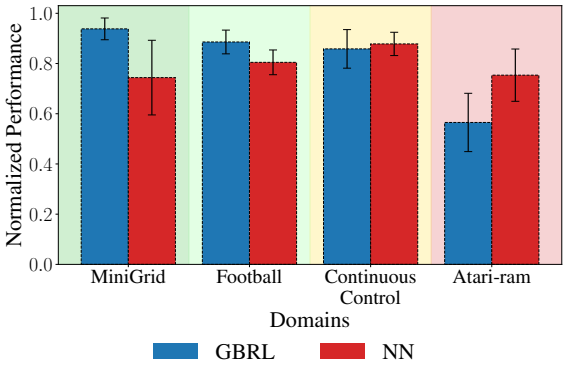


Figure 4. **GBRL vs NN in standard environments (PPO).** Aggregated mean and standard deviation of the normalized average reward for the final 100 episodes.

Decision trees, at their core, are if-else structures that excel in handling numerical and categorical data. In MiniGrid, which provides such environments, GBRL significantly outperformed NN. For example, in tasks like PutNear, FourRooms, and Fetch, GBRL consistently achieved higher rewards, highlighting its ability to exploit the structured nature of the environment effectively. This performance demonstrates that GBRL is particularly well-suited for problems where data can be neatly partitioned, aligning with GBT's inherent design strengths. This suggests that structured environments represent an ideal use case for GBRL.

An additional domain characterized by structured representations is the Football domain, where we used a shaped reward. Here, features such as player positions and ball locations were designed to represent identifiable, interpretable information. In this domain, GBRL outperformed NN across most scenarios and exhibited equivalent performance in the rest. The structured nature of the Football domain aligns well with the strengths of GBT, effectively partitioning the feature space. Interestingly, while both Football and MiniGrid are structured, the Football domain is not sparse. Hence, GBRL's strong performance in Football underscores its robustness and capacity to generalize well in structured, high-dimensional environments.

Unlike Football, the Atari RAM domain offers unstructured, high-dimensional representations by exposing raw system memory states. These flattened views lack explicit semantic structure, making them challenging for GBRL, which relies on single-feature splits. As a result, GBRL underperformed compared to NN in most Atari RAM tasks. This

result highlights an important limitation of decision-tree-based approaches. They struggle to model implicit feature interactions, an inherent characteristic of unstructured data.

Finally, classic control tasks are simpler environments that rely on low-dimensional state spaces. In these tasks, GBRL demonstrated performance comparable to that of the NN counterpart. This suggests that decision tree-based models can match NNs in solving tasks where feature interactions are limited and the optimization landscape is smooth.

Overall, these comparisons indicate that GBT can serve as a strong function approximator in RL, matching or surpassing NN's performance in certain environments.

5.2. Challenge Environments

Having established GBRL's general capabilities, we now examine scenarios designed to test inherent properties of decision trees. These experiments focus on challenging conditions—sparse rewards, misleading correlations, and input perturbations—where NNs typically struggle.

5.2.1. Signal Dilution

Signal dilution occurs when rare events, such as sparse rewards, are statistically overwhelmed by more common non-event data during training (Shyalika et al., 2024; Zhao et al., 2018; He & Cheng, 2021; Tessler et al., 2017). In NNs with fixed architectures, shared weights inherently prioritize patterns from dominant classes or frequent signals, causing rare events to be "averaged out" or under-represented in learned representations. This phenomenon is particularly problematic in RL and classification tasks with imbalanced datasets, where the model fails to retain or react to critical but infrequent signals.

Experimental Setup. We compare GBRL with NNs in a linear equation variable isolation environment with sparse rewards, inspired by (Poesia et al., 2021). The agent observes the coefficients (a, b, c) of a linear equation $ax + b = c$, and needs to isolate x, receiving a large reward only upon completion. Actions are multi-discrete, involving the selection of an operation, a digit $\in [0, 9]$, and its sign. The optimal policy solves the simple task in 2 steps. We also evaluate two more complex variations with an additional variable y, where the optimal policy requires 3–4 steps. For these, we introduce intermediate goals with small rewards to guide the agent.

Results. While simple for humans, these tasks remain challenging for NNs due to signal dilution (Figure 5). Although they can make some progress in the simple task ($ax + b = c$), they fail when the equation complexity increases, even when given intermediate rewards. In contrast, GBRL converges to and retains a stable (and optimal) policy throughout the process. GBTs naturally segment the decision space into explicit rule-based splits, preventing signal dilution while correctly preserving the rare but crucial reward signals. Consequently, we see that GBRL can preserve and exploit rare but critical reward signals—evidence that GBT-based methods are inherently less prone to signal dilution.

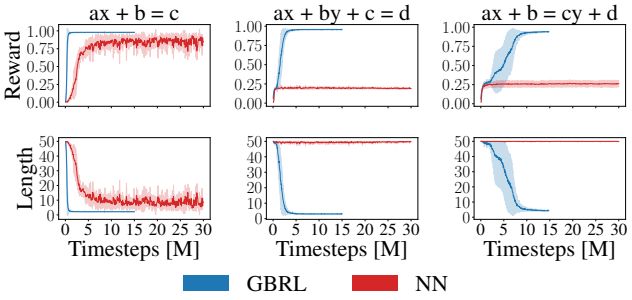


Figure 5. **Signal dilution, variable isolation task.** Mean and standard deviation of the average episodic reward during training. GBRL was trained for 15M training steps and NN for 30M. Episodes are terminated after 50 steps if not solved.

5.2.2. Spurious Correlations

Experimental Setup. The agent is placed in a grid environment with three balls (red, green, and blue), and a red box. The ball locations are randomized. The agent observes an objective specifying which ball to pick up. We experiment with three box-placement configurations: adjacent to the target ball, adjacent to one of the non-target balls, and without a box in the room. This task tests whether the policy overfits spuriously correlated features. We then analyze the results using SHAP (Lundberg & Lee, 2017), a method based on Shapley values (ARROW et al., 1953) that measures the contribution of each feature to the final prediction of a model.

Results. GBRL consistently outperforms NNs in all scenarios (Figure 6), demonstrating its ability to ignore irrelevant distractions, such as the box, and reliably reach the target regardless of its placement.

To understand GBRL's performance, we analyzed the SHAP values for the agent's surroundings and the mission feature when predicting the logit for picking up the target object (Figure 7). GBRL assigns the strongest positive SHAP value to the target object, followed by the mission string, aligning perfectly with the goal. Non-target objects receive the most negative SHAP values, suppressing distractions and reinforcing goal-directed behavior. In contrast, NNs assign negative SHAP values to the mission feature and rely heavily on the grid surrounding the target object. This indicates a reliance on spurious correlations and a lack of

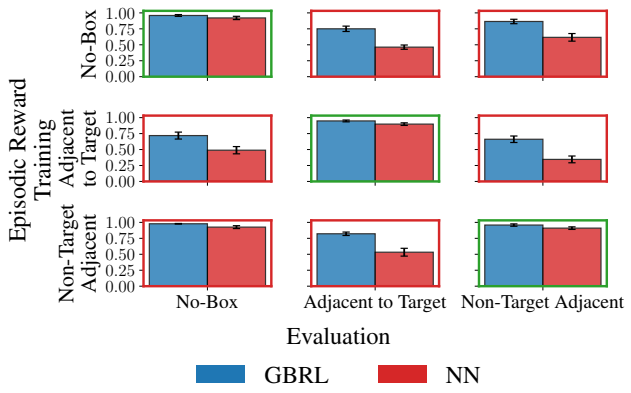


Figure 6. **Spurious correlations, MiniGrid.** Mean and standard deviation of the episodic reward. Plots outlined in green are in-distribution, while plots outlined in red are out-of-distribution.

goal-oriented strategies.

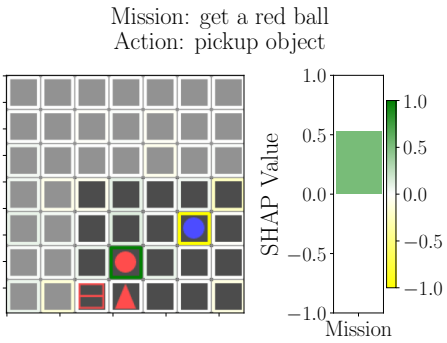
These findings underscore GBRL's ability to focus on task-relevant features and ignore deceptive cues, reinforcing our hypothesis that GBT's inductive bias provides robustness against spurious correlations. Next, we consider how this robustness extends to other forms of state perturbations.

5.2.3. Robustness to State Perturbations

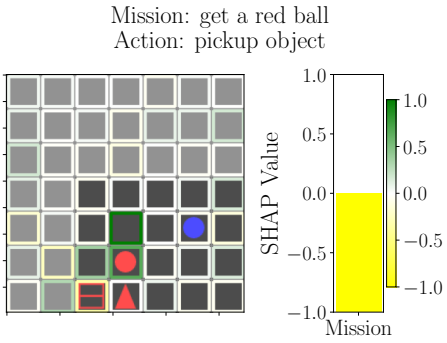
When encountering an out-of-distribution (OOD) state, a GBT model will always output a result within the same range seen in training. In contrast, NNs may extrapolate and exhibit unexpected behavior (Xu et al., 2021; Meinke & Hein, 2020; Ulmer & Cinà, 2021). We consider three tasks. First, robustness to irrelevant information. Second, robustness to random noise. And finally, robustness to missing features. These experiments aim to test the inherent robustness properties of GBTs when encountering different types of noise during training.

Irrelevant Information. We consider a similar fetch experiment as before (Section 5.2.2). Now, a new object is placed in the room at a random location. This object is not part of the mission and only interferes with navigation. The interfering object differs from the task objects in shape, color, or both.

Figure 8 illustrates how introducing a distractor impacts policy performance. When the environment is free of distracting objects, NNs are able to solve the task, converging to a stable and ideal solution. Surprisingly, although the purple-box object shares no features with the task objects (blue, green, and red balls), NNs exhibit a drop in performance but are able to solve the task in a subset of seeds. This performance degradation increases when the confusing object shares more features with the goal objects, such as purple-*ball* and *red*-box. In contrast, GBRL successfully



(a) **GBRL SHAP.**



(b) **NN SHAP.**

Figure 7. **Spurious correlations revealed by feature importance in MiniGrid.** The visualization shows normalized SHAP values for the concluding action in a successful evaluation run, revealing the magnitude and direction of each feature's impact on model predictions. In this scenario, a red box was positioned next to the target (red ball), and both models converged to a successful policy. GBRL prioritizes the mission, ignores the red box, and contrasts the blue ball. In contrast, the NN focuses on the cells surrounding the target, while ignoring the blue ball and contrasting the box and the mission.

and efficiently solves the task, regardless of the additional irrelevant object placed in the scene.

Random Noise. In the classic-control environments (Section 5.1), we train agents in a noiseless setting. Then, to evaluate the robustness of the resulting policies, we add random zero-centered Gaussian noise to the input states as defined by: $s_t = s_t + \epsilon |s_t|$.

The results show that these characteristics translate to practical outcomes (Figure 9). In classic-control environments, a trained GBT-based policy is much more robust to small variations in the state space compared to NNs. We attribute this performance to how GBTs partition the input space. This partitioning occurs on hard thresholds that are split according to single features. Although small perturbations may be sufficient to completely change decision outcomes in NNs (Lütjens et al., 2020; Carlini & Wagner, 2017), GBRL

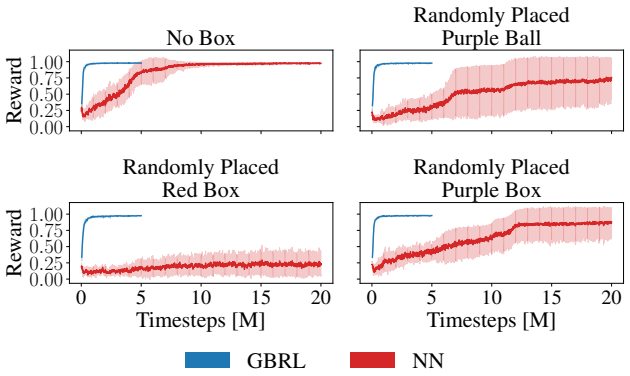


Figure 8. **Robustness to irrelevant objects, MiniGrid.** Mean and standard deviation of the average episodic reward during training. GBRL was trained for 5M training steps, and NN for 20M.

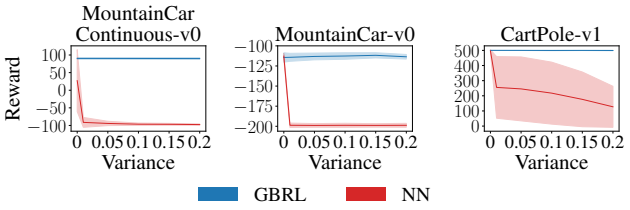


Figure 9. **Robustness to input perturbations, classic-control.** Models were trained without perturbations and then evaluated on noisy observations. At each timestep, the observation was perturbed following $s_t = s_t + \epsilon |s_t|$, where $\epsilon \sim \mathcal{N}(0, \text{var})$. We report the mean and standard deviation of episodic reward across 100 evaluation episodes for each Variance $\in [0, 0.2]$.

exhibits robustness to random additive noise.

Missing Features. We modify scenarios in the football domain (Section 5.1) to support the random drop of a player from the opposing team. Policies are trained on both the original task and a modified variant. We then evaluate model robustness by testing how a policy trained against N random players performs when faced with $N + 1$, or how a policy trained against a full team behaves when a random opponent is removed.

The results show that GBRL outperforms NNs in OOD tasks (Figure 10), particularly in the 11 vs 11 academy scenario. Notably, GBRL maintains performance when evaluated against 10 opponents and can solve 11 vs. 11 even when trained on 10. In contrast, NNs struggle in all OOD cases. This suggests GBRL better isolates features that are essential to solving the task.

6. Ablation Study

In this section, we examine the sensitivity of GBRL to key hyperparameters. We systematically changed one hyper-

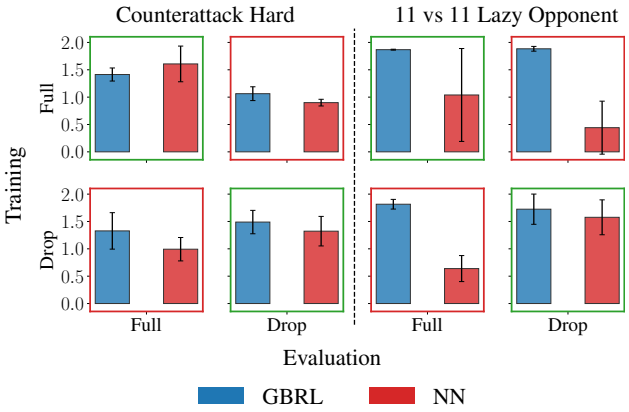


Figure 10. **Out-of-distribution evaluation, football.** Mean and standard deviation of episodic reward across 100 evaluation episodes. Plots outlined in green are in-distribution, while plots outlined in red are out-of-distribution.

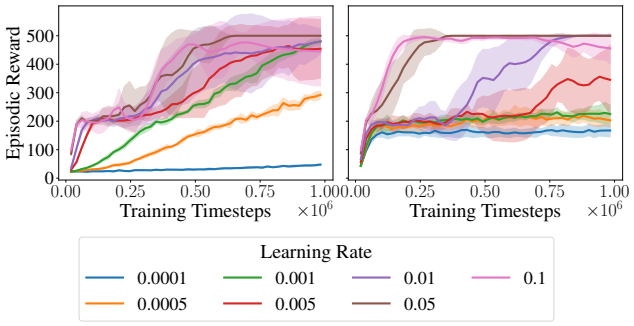


Figure 11. **Ablation: Learning rate variation across policy (left) and value function (right) components**. Mean and standard deviation of the average episodic reward during training. Each subplot shows the effect of varying the learning rate for one component (policy or value function) while keeping the other fixed.

parameter at a time while keeping the others fixed. For each experiment, we trained an agent in the CartPole-v1 environment for 1M timesteps using GBT-based PPO.

Learning Rate Variation Across Policy and Value Function Components. We investigated the impact of the learning rate for both the policy and value function components. In these experiments, we held one learning rate constant while varying the other. Consistent with observations in NN training, we found that excessively high learning rates destabilized the learning process, whereas overly low rates led to significantly slower convergence (as illustrated in Figure 11).

Limitations of Tree Depth. We varied the maximum depth of the decision trees built at each training iteration and evaluated their impact on learning dynamics. Figure 12 demonstrates that deeper trees yield more accurate gradi-

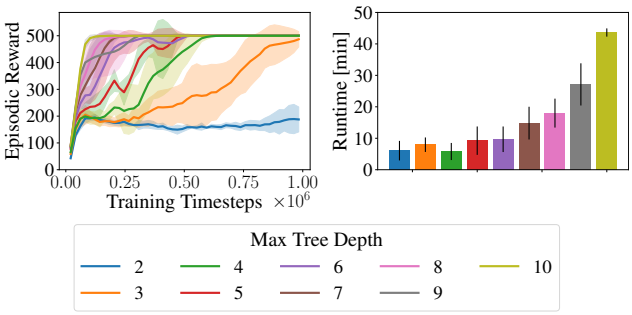


Figure 12. **Ablation: Effect of tree depth on convergence**. We varied the maximum depth of the decision trees used in the gradient-boosted models and evaluated their impact on learning dynamics. The plot shows the mean and standard deviation of the average episodic reward over time and the total runtime.

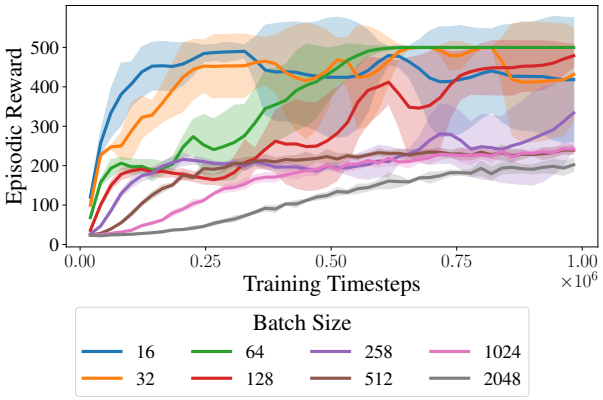


Figure 13. **Ablation: Batch size impact on convergence**. Mean and standard deviation of the average episodic reward during training. We fix the rollout length to 2048 and vary the batch size. Smaller batches improve early learning but suffer from instability, while larger batches stabilize training at the cost of slower convergence.

ent approximations, leading to faster convergence, but also incur a higher computational cost per training step. In the paper, we selected a maximum tree depth of 4, balancing convergence speed and wall-clock time.

Batch Size Impact on Convergence. We analyzed the impact of the batch size on performance. In Figure 13, we observe that batch size significantly impacts convergence. Specifically, smaller batches result in GBRL building more trees per rollout, improving adaptability. However, smaller batches also lead to noisier gradient estimates as a result of limited samples per constructed tree. Conversely, larger batches stabilize training by reducing variance through averaging within leaves, as more samples are utilized to construct each tree but build fewer trees per rollout. Hence, both excessively small and large batch sizes negatively impact performance.

Overall, these ablations highlight that GBRL's performance is sensitive to training frequency, tree depth, and batch size—each of which governs the trade-off between convergence speed and stability. While this section focused on general hyperparameters, we further investigate how these trade-offs manifest across different RL algorithms in Section B.

7. Limitations and Future Directions

GBRL exhibits competitive performance and surprising robustness capabilities. However, despite these benefits, we highlight two important limitations. **(i) Continuous generation of trees.** As the policy improves through numerous updates, the size of the ensemble increases. This unbounded growth has implications for memory usage, computational efficiency, and the feasibility of online real-time adaptation. Future work may leverage tree redundancy to produce a more efficient and compact policy. For example, strategies for tree pruning, ensemble compression, or dynamically managing ensemble size. **(ii) Off-policy continuous control.** Methods like DDPG (Lillicrap et al., 2019) and SAC (Haarnoja et al., 2018) update the policy by differentiating the Q-estimator. As GBTs are not differentiable, new solutions are needed to incorporate them into these algorithms.

8. Conclusion

Historically, RL practitioners have relied on tabular, linear, and NN-based function approximators. While GBT evolved as a widely successful tool in supervised learning, it has been absent from this toolbox. In this work, we introduced a method for effectively integrating GBT into RL. We demonstrated domains where GBT excels compared to NNs, in addition to analyzing its various inherent characteristics.

We observe that the ideal method depends on the task characteristics. Tabular and linear approaches are suitable for small state spaces or simple mappings, while NNs excel when tackling complex relationships in unstructured data. In comparison, GBT thrives in complex, yet structured environments. In such cases, GBRL's advantage is highlighted, reflecting GBTs known benefits from supervised learning.

A crucial component of GBRL is our efficient adaptation of GBT for Actor-Critic methods, which allows the simultaneous optimization of distinct objectives. We optimized this approach for large-scale ensembles using GPU acceleration (CUDA). Furthermore, GBRL integrates seamlessly with existing RL libraries, promoting ease of use and adoption.

GBRL is a step toward well-suited solutions for real-world tasks with structured data.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Abel, D., Agarwal, A., Diaz, F., Krishnamurthy, A., and Schapire, R. E. Exploratory gradient boosting for reinforcement learning in complex domains, 2016. URL https://arxiv.org/abs/1603.04119.

Andrychowicz, M., Raichuk, A., Stanczyk, P., Orsini, M., Girgin, S., Marinier, R., Hussenot, L., Geist, M., Pietquin, O., Michalski, M., Gelly, S., and Bachem, O. What matters in on-policy reinforcement learning? a large-scale empirical study, 2020. URL https://arxiv.org/abs/2006.05990.

Arik, S. O. and Pfister, T. Tabnet: Attentive interpretable tabular learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8):6679–6687, May 2021. doi: 10.1609/aaai.v35i8.16826. URL https://ojs.aaai.org/index.php/AAAI/article/view/16826.

ARROW, K. J., BARANKIN, E. W., BLACKWELL, D., BOTT, R., DALKEY, N., DRESHER, M., GALE, D., GILLIES, D. B., GLICKSBERG, I., GROSS, O., KARLIN, S., KUHN, H. W., MAYBERRY, J. P., MILNOR, J. W., MOTZKIN, T. S., NEUMANN, J. V., RAIFFA, H., SHAPLEY, L. S., SHIFFMAN, M., STEWART, F. M., THOMPSON, G. L., and THRALL, R. M. *Contributions to the Theory of Games (AM-28), Volume II*. Princeton University Press, 1953. ISBN 9780691079356. URL http://www.jstor.org/stable/j.ctt1b9x1zv.

Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, June 2013. ISSN 1076-9757. doi: 10.1613/jair.3912. URL http://dx.doi.org/10.1613/jair.3912.

Beyazit, E., Kozaczuk, J., Li, B., Wallace, V., and Fadlallah, B. An inductive bias for tabular deep learning. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 43108–43135. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/8671b6dffc08b4fcf5b8ce26799b2bef-Paper-Conference.pdf.

Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., and Kasneci, G. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35:7499–7519, 2021. URL https://api.semanticscholar.org/CorpusID:238353897.

Brukhim, N., Hazan, E., and Singh, K. A boosting approach to reinforcement learning. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 33806–33817. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/daf8364f0715a41a469c677c0adc4754-Paper-Conference.pdf.

Carlini, N. and Wagner, D. Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, AISec '17, pp. 3–14, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450352024. doi: 10.1145/3128572.3140444. URL https://doi.org/10.1145/3128572.3140444.

Chen, T. Machine learning challenge winning solutions, 2023. https://github.com/dmlc/xgboost/tree/master/demo#machine-learning-challenge-winning-solutions.

Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16. ACM, August 2016. doi: 10.1145/2939672.2939785. URL http://dx.doi.org/10.1145/2939672.2939785.

Chevalier-Boisvert, M., Dai, B., Towers, M., Perez-Vicente, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 73383–73394. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/e8916198466e8ef218a2185a491b49fa-Paper-Datasets_and_Benchmarks.pdf.

Duan, T., Anand, A., Ding, D. Y., Thai, K. K., Basu, S., Ng, A., and Schuler, A. NGBoost: Natural gradient boosting for probabilistic prediction. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*,

volume 119 of *Proceedings of Machine Learning Research*, pp. 2690–2700. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/duan20a.html.

Friedman, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189 – 1232, 2001. doi: 10.1214/aos/1013203451. URL https://doi.org/10.1214/aos/1013203451.

Gorishniy, Y., Rubachev, I., Khrulkov, V., and Babenko, A. Revisiting deep learning models for tabular data. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 18932–18943. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/9d86d83f925f2149e9edb0ac3b49229c-Paper.pdf.

Gorishniy, Y., Rubachev, I., and Babenko, A. On embeddings for numerical features in tabular deep learning. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 24991–25004. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9e9f0ffc3d836836ca96cbf8fe14b105-Paper-Conference.pdf.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. Why do tree-based models still outperform deep learning on typical tabular data? In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 507–520. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/0378c7692da36807bdec87ab043cdadc-Paper-Datasets_and_Benchmarks.pdf.

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/haarnoja18b.html.

Hancock, J. T. and Khoshgoftaar, T. M. Survey on categorical data for neural networks. *Journal of Big Data*, 7(1): 28, Apr 2020. ISSN 2196-1115. doi: 10.1186/s40537-020-00305-w. URL https://doi.org/10.1186/s40537-020-00305-w.

He, J. and Cheng, M. X. Weighting methods for rare event identification from imbalanced datasets. *Frontiers in big Data*, 4:715320, 2021. doi: 10.3389/fdata.2021.715320. URL https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2021.715320.

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi: 10.1609/aaai.v32i1.11694. URL https://ojs.aaai.org/index.php/AAAI/article/view/11694.

Hollmann, N., Müller, S., Eggensperger, K., and Hutter, F. Tabpfn: A transformer that solves small tabular classification problems in a second, 2023. URL https://arxiv.org/abs/2207.01848.

Ivanov, S. and Prokhorenkova, L. Boost then convolve: Gradient boosting meets graph neural networks, 2021. URL https://arxiv.org/abs/2101.08543.

Jeffares, A., Curth, A., and van der Schaar, M. Deep learning through a telescoping lens: A simple model provides empirical insights on grokking, gradient boosting & beyond. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 123498–123533. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/df334022279996b07e0870a629c18857-Paper-Conference.pdf.

Kadra, A., Lindauer, M., Hutter, F., and Grabocka, J. Well-tuned simple nets excel on tabular datasets. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 23928–23941. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/c902b497eb972281fb5b4e206db38ee6-Paper.pdf.

Katzir, L., Elidan, G., and El-Yaniv, R. Net-dnf: Effective deep modeling of tabular data. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=73WTGs96kho.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. Lightgbm: A highly efficient gradient boosting decision tree. In Guyon, I., Luxburg, U. V.,

Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf.

Kersting, K. and Driessens, K. Non-parametric policy gradients: a unified treatment of propositional and relational domains. In *Proceedings of the 25th International Conference on Machine Learning*, ICML '08, pp. 456–463, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781605582054. doi: 10.1145/1390156.1390214. URL https://doi.org/10.1145/1390156.1390214.

Kurach, K., Raichuk, A., Stanczyk, P., Zajac, M., Bachem, O., Espeholt, L., Riquelme, C., Vincent, D., Michalski, M., Bousquet, O., and Gelly, S. Google research football: A novel reinforcement learning environment. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04):4501–4510, Apr. 2020. doi: 10.1609/aaai.v34i04.5878. URL https://ojs.aaai.org/index.php/AAAI/article/view/5878.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning, 2019. URL https://arxiv.org/abs/1509.02971.

London, B., Lu, L., Sandler, T., and Joachims, T. Boosted off-policy learning. In Ruiz, F., Dy, J., and van de Meent, J.-W. (eds.), *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pp. 5614–5640. PMLR, 25–27 Apr 2023. URL https://proceedings.mlr.press/v206/london23a.html.

Lundberg, S. M. and Lee, S.-I. A unified approach to interpreting model predictions. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30*, pp. 4765–4774. Curran Associates, Inc., 2017. URL http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf.

Lütjens, B., Everett, M., and How, J. P. Certified adversarial robustness for deep reinforcement learning. In Kaelbling, L. P., Kragic, D., and Sugiura, K. (eds.), *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1328–1337. PMLR, 30 Oct–01 Nov 2020. URL https://proceedings.mlr.press/v100/lutjens20a.html.

Lyzhin, I., Ustimenko, A., Gulin, A., and Prokhorenkova, L. Which tricks are important for learning to rank? In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 23264–23278. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/lyzhin23a.html.

Ma, H., Cao, J., Fang, Y., Zhang, W., Sheng, W., Zhang, S., and Yu, Y. Retrieval-based gradient boosting decision trees for disease risk assessment. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pp. 3468–3476, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393850. doi: 10.1145/3534678.3539052. URL https://doi.org/10.1145/3534678.3539052.

Malinin, A., Prokhorenkova, L., and Ustimenko, A. Uncertainty in gradient boosting via ensembles, 2021. URL https://arxiv.org/abs/2006.10562.

Mason, L., Baxter, J., Bartlett, P., and Frean, M. Boosting algorithms as gradient descent. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/96a93ba89a5b5c6c226e49b88973f46e-Paper.pdf.

McElfresh, D., Khandagale, S., Valverde, J., Prasad C, V., Ramakrishnan, G., Goldblum, M., and White, C. When do neural nets outperform boosted trees on tabular data? In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 76336–76369. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f06d5ebd4ff40b40dd97e30cee632123-Paper-Datasets_and_Benchmarks.pdf.

Meinke, A. and Hein, M. Towards neural networks that provably know when they don't know, 2020. URL https://arxiv.org/abs/1909.12180.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Balcan, M. F. and Weinberger, K. Q. (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1928–1937, New York, New

York, USA, 20–22 Jun 2016. PMLR. URL https://proceedings.mlr.press/v48/mniha16.html.

NVIDIA. *CUDA Toolkit Documentation*, 2025. URL https://developer.nvidia.com/cuda-toolkit. Version 12.1.

Ota, K., Jha, D. K., and Kanezaki, A. A framework for training larger networks for deep reinforcement learning. *Mach. Learn.*, 113(9):6115–6139, June 2024. ISSN 0885-6125. doi: 10.1007/s10994-024-06547-6. URL https://doi.org/10.1007/s10994-024-06547-6.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library, 2019. URL https://arxiv.org/abs/1912.01703.

Peng, X. B., Kumar, A., Zhang, G., and Levine, S. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning, 2019. URL https://arxiv.org/abs/1910.00177.

Poesia, G., Dong, W., and Goodman, N. Contrastive reinforcement learning of symbolic reasoning domains. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 15946–15956. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/859555c74e9afd45ab771c615c1e49a6-Paper.pdf.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. Catboost: unbiased boosting with categorical features. In Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/14491b756b3a51daac41c24863285549-Paper.pdf.

Raffin, A. Rl baselines3 zoo. https://github.com/DLR-RM/rl-baselines3-zoo, 2020.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL http://jmlr.org/papers/v22/20-1364.html.

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. In Bengio, Y. and LeCun, Y. (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL http://arxiv.org/abs/1506.02438.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Seto, H., Oyama, A., Kitora, S., Toki, H., Yamamoto, R., Kotoku, J., Haga, A., Shinzawa, M., Yamakawa, M., Fukui, S., and Moriyama, T. Gradient boosting decision tree becomes more reliable than logistic regression in predicting probability for diabetes with big data. *Scientific Reports*, 12(1):15889, Oct 2022. ISSN 2045-2322. doi: 10.1038/s41598-022-20149-z. URL https://doi.org/10.1038/s41598-022-20149-z.

Shwartz-Ziv, R. and Armon, A. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. ISSN 1566-2535. doi: https://doi.org/10.1016/j.inffus.2021.11.011. URL https://www.sciencedirect.com/science/article/pii/S1566253521002360.

Shyalika, C., Wickramarachchi, R., and Sheth, A. P. A comprehensive survey on rare event prediction. *ACM Comput. Surv.*, 57(3), November 2024. ISSN 0360-0300. doi: 10.1145/3699955. URL https://doi.org/10.1145/3699955.

Sigrist, F. Gaussian process boosting. *Journal of Machine Learning Research*, 23(232):1–46, 2022. URL http://jmlr.org/papers/v23/20-322.html.

Somepalli, G., Goldblum, M., Schwarzschild, A., Bruss, C. B., and Goldstein, T. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training, 2021. URL https://arxiv.org/abs/2106.01342.

Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018. URL http://incompleteideas.net/book/the-book-2nd.html.

Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In Solla, S., Leen, T., and Müller, K. (eds.), *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/

464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

Tessler, C., Givony, S., Zahavy, T., Mankowitz, D., and Mannor, S. A deep hierarchical approach to lifelong learning in minecraft. *Proceedings of the AAAI Conference on Artificial Intelligence*, 31(1), Feb. 2017. doi: 10.1609/aaai.v31i1.10744. URL https://ojs.aaai.org/index.php/AAAI/article/view/10744.

Tian, Z., Xiao, J., Feng, H., and Wei, Y. Credit risk assessment based on gradient boosting decision tree. *Procedia Computer Science*, 174:150–160, 2020. ISSN 1877-0509. doi: https://doi.org/10.1016/j.procs.2020.06.070. URL https://www.sciencedirect.com/science/article/pii/S1877050920315842. 2019 International Conference on Identification, Information and Knowledge in the Internet of Things.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., Cola, G. D., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., and Younis, O. G. Gymnasium: A standard interface for reinforcement learning environments, 2024. URL https://arxiv.org/abs/2407.17032.

Ulmer, D. and Cinà, G. Know your limits: Uncertainty estimation with relu classifiers fails at reliable ood detection. In de Campos, C. and Maathuis, M. H. (eds.), *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pp. 1766–1776. PMLR, 27–30 Jul 2021. URL https://proceedings.mlr.press/v161/ulmer21a.html.

Ustimenko, A. and Prokhorenkova, L. Stochasticrank: Global optimization of scale-free discrete functions. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 9669–9679. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/ustimenko20a.html.

Ustimenko, A., Beliakov, A., and Prokhorenkova, L. Gradient boosting performs gaussian process inference, 2023. URL https://arxiv.org/abs/2206.05608.

Wassan, S., Suhail, B., Mubeen, R., Raj, B., Agarwal, U., Khatri, E., Gopinathan, S., and Dhiman, G. Gradient boosting for health iot federated learning. *Sustainability*, 14(24), 2022. ISSN 2071-1050. doi: 10.3390/su142416842. URL https://www.mdpi.com/2071-1050/14/24/16842.

Xu, K., Zhang, M., Li, J., Du, S. S., ichi Kawarabayashi, K., and Jegelka, S. How neural networks extrapolate: From feedforward to graph neural networks, 2021. URL https://arxiv.org/abs/2009.11848.

Zabërgja, G., Kadra, A., and Grabocka, J. Tabular data: Is attention all you need?, 2024. URL https://arxiv.org/abs/2402.03970.

Zhao, Y., Wong, Z. S.-Y., and Tsui, K. L. A framework of rebalancing imbalanced healthcare data for rare events' classification: A case of look-alike sound-alike mix-up incident detection. *Journal of Healthcare Engineering*, 2018(1):6275435, 2018. doi: https://doi.org/10.1155/2018/6275435. URL https://onlinelibrary.wiley.com/doi/abs/10.1155/2018/6275435.

Appendix

This appendix provides supplementary materials that support the findings and methodologies discussed in the main text. It is organized into five sections to present the full experiment results, implementation details, hyperparameters used during the experiments, training progression plots, and experimental plots, respectively. These materials offer detailed insights into the research process and outcomes, facilitating a deeper understanding and replication of the study.

A. Multi-Objective Training With GBT

In GBRL, we compute gradients for both the actor and critic objectives per timestep. We then concatenate these gradients: $g_t := [g_{t,\text{actor}}, g_{t,\text{critic}}]$ and use them to build a decision tree. At each candidate node split, we evaluate a score function to decide the best split. However, since the actor and critic gradients can differ in magnitude, this score can become biased toward one objective. To address this, we explored two strategies:

- Gradient normalization per output dimension with L2-based split scoring.
- Cosine similarity, which emphasizes gradient direction over magnitude.

Both approaches are supported in our codebase and produce good results, while the cosine similarity performed slightly better (Figure 14).

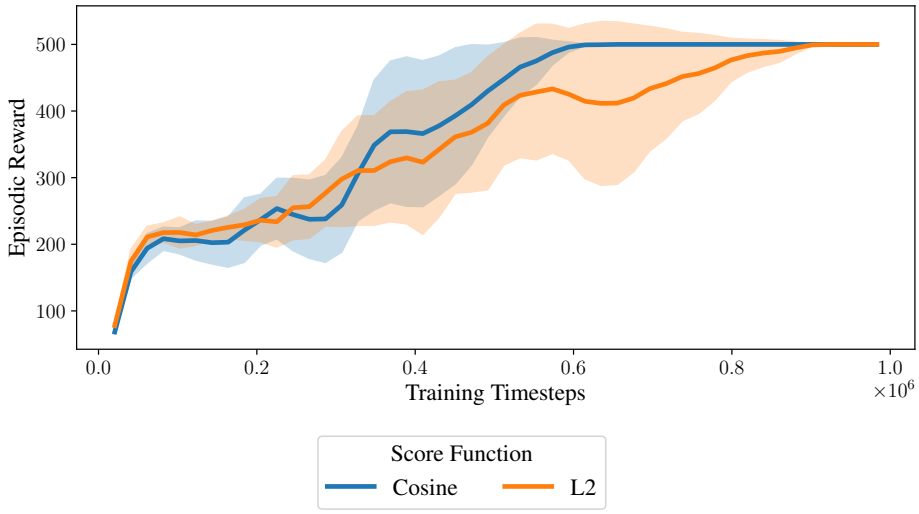


Figure 14. **Multi-objective gradient split strategies in PPO-GBRL on CartPole-v1**. We compare gradient normalization and cosine similarity as methods for handling actor-critic gradient imbalance when building trees. Both strategies yield good performance, with cosine similarity achieving slightly faster convergence. Curves show average episodic reward over 1M timesteps.

B. Why Do Certain Algorithms Perform Differently Than Others?

Apart from inherent algorithmic differences, the main difference for GBRL is the number of fitted trees per update. This is analogous to the number of update steps in a neural network, since each tree represents a functional gradient step for the actor or critic. To illustrate this effect, we trained agents in the CartPole-v1 environment for 1M timesteps and varied the number of epochs while keeping the batch size and the rollout length fixed, thereby varying the number of updates per rollout. We repeated this experiment for several batch sizes while keeping the rollout length fixed at 2048. As shown in Figure 15, increasing the number of trees added per rollout improves performance, but increases computational cost. Moreover, a larger batch size also increases computational time, due to a larger number of samples used to build each tree. Hence, there is a clear trade-off between computational cost and performance.

This trade-off also explains the difference in performance of various algorithms when combined with GBRL, as their update schemes inherently affect how many trees are fitted per rollout.

- **A2C** - performs a single gradient step per rollout using the entire batch. In GBRL, this results in a single constructed tree per rollout. Since GBTs update the value and policy functions directly, such large, coarse updates can destabilize learning.
- **PPO** - in contrast, uses minibatches and multiple epochs per rollout to better control the trade-off. This allows GBRL to fit several small trees per rollout, yielding more stable and incremental updates.
- **AWR** - is an off-policy algorithm. Therefore, GBRL could potentially build many trees without adding sample complexity. However, we noticed that our best performing models had a huge computational cost to finish training. As a result, we resorted to reducing the number of gradient steps per update at the expense of performance. Future work could investigate methods for optimizing GBRL with a fixed tree budget (such as pruning and distillation), which may enable the applicability of more demanding RL algorithms, such as AWR, using GBRL.

In general, as each tree approximates a gradient step, GBRL benefits from frequent updates to refine its ensemble gradually. This makes it particularly well-suited to algorithms that support minibatching and multiple epochs per rollout.

C. Implementation Details and Hyperparameters

This section provides implementation details, information regarding compute resources, and tables containing the hyperparameters used in our experiments enabling the reproducibility of our results. Table 2 lists GBRL hyperparameters for all experiments.

C.1. Algorithm Details

Our GBRL method is implemented within the Stable Baselines3 framework (Raffin et al., 2021), a widely used library for standardized reinforcement learning algorithms. For comparisons, we use baseline NNs from RL Zoo3 (Raffin, 2020), which provides tuned hyperparameters per environment. For all experiments, we used a standard MLP with two hidden layers, each containing 64 units and tanh activations. The hidden layers are shared between the actor and the critic, and each component has an additional separate linear layer. This architecture is the default in Stable Baselines3 for the environments we tested and is widely used as a baseline in RL literature. We maintained this consistent architecture across all environments to ensure fair comparison.

A2C (Mnih et al., 2016) is a synchronous, on-policy Actor-Critic algorithm designed to improve learning stability. The critic learns a value function, $V(\mathrm{s})$, used to estimate the advantage. This advantage is incorporated into the policy gradient updates, reducing variance and leading to smoother learning. The policy is updated using the following gradient: $\nabla_\theta J(\pi_\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(\mathrm{a} \,|\, \mathrm{s}) A(\mathrm{s}, \mathrm{a})]$.

AWR (Peng et al., 2019) is an off-policy Actor-Critic algorithm. Provided a dataset $\mathcal{D}$, AWR updates both the policy and the value through supervised learning. This dataset can be pre-defined and fixed (offline), or continually updated using the agent's experience (replay buffer). At each training iteration k, AWR solves the following two regression problems:

$$V_k = \arg\min_V \mathbb{E}_{\mathrm{s},\mathrm{a}\sim\mathcal{D}}[\|G(\mathrm{s},\mathrm{a}) - V(\mathrm{s})\|_2^2] \,,$$

$$\pi_{k+1} = \arg\max_\pi \mathbb{E}_{\mathrm{s},\mathrm{a}\sim\mathcal{D}}[\log \pi(\mathrm{a} \,|\, \mathrm{s}) \exp(\frac{1}{\beta} A_k(\mathrm{s},\mathrm{a}))] \,,$$

where $G(\mathrm{s},\mathrm{a})$ represents the monte-carlo estimate or TD(λ) of the expected return (Sutton & Barto, 2018).

Advantage computation: We use Generalized Advantage Estimation (GAE) as proposed by Schulman et al. (2016) to compute the advantage: $A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$, where $\delta_t = r_t + \gamma V(\mathrm{s}_{t+1}) - V(\mathrm{s}_t)$. The GAE parameters and for each environment are provided in Table 2 of the appendix. Critic: We use on-policy actor-critic algorithms, where the critic predicts the value function (not action-value Q). We use a standard L2 loss between the critic's prediction and the target return estimate: $L_{\text{critic}}(\theta) = ||V_\theta(\mathrm{s}_t) - G_t||^2$, where G_t is either a Monte Carlo return estimate or a bootstrapped n-step return, depending on the specific algorithm (PPO, A2C, or AWR).

C.2. Environments

Classic-Control & Box2D. We evaluate GBRL against NNs on all available classic-control tasks provided in Gymnasium (Towers et al., 2024) and on LunarLander-v2. This includes three discrete action space environments: Acrobot-v1, CartPole-

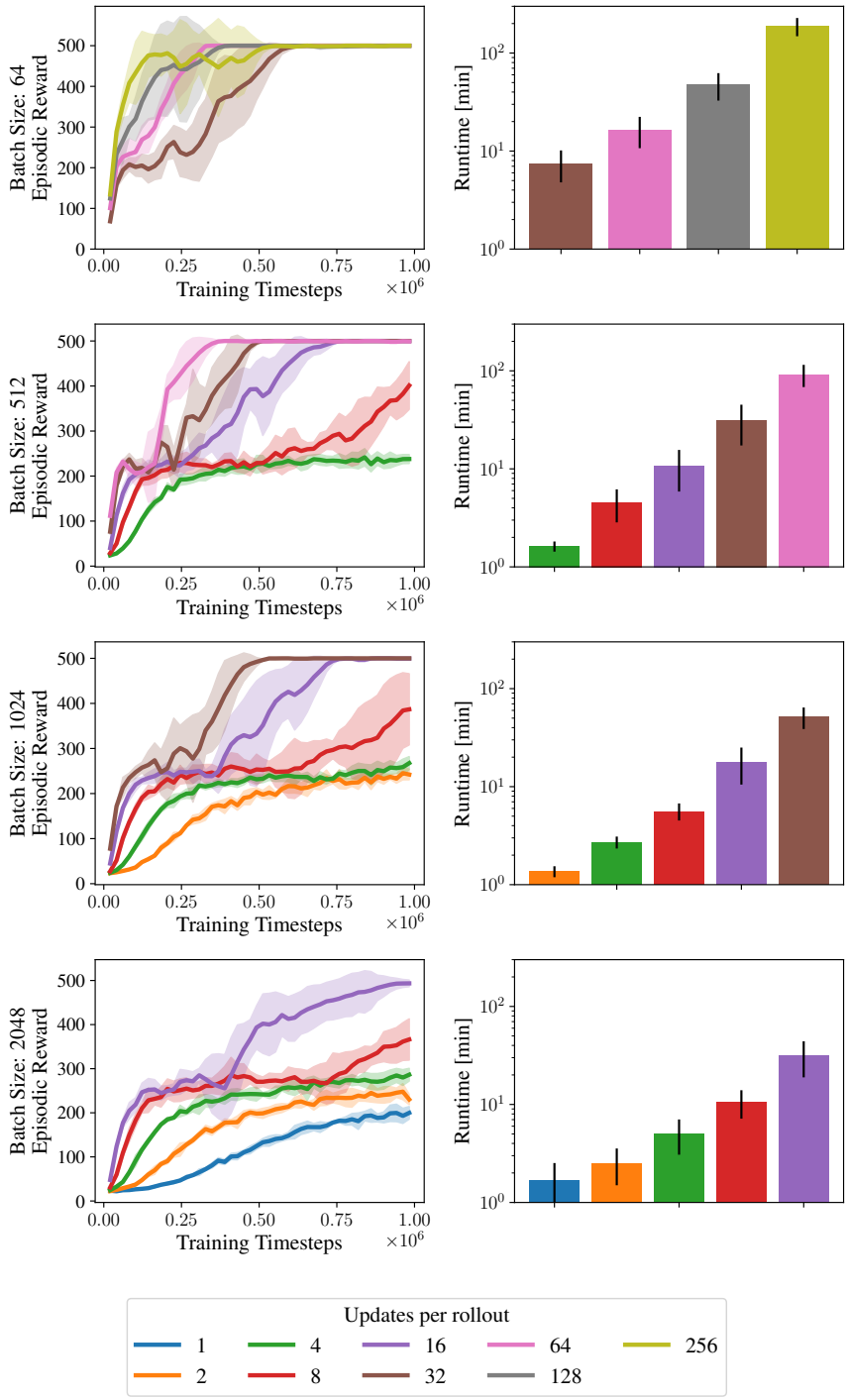


Figure 15. **Effect of number of gradient updates per rollout on PPO-GBRL's performance in CartPole-v1**. We fix the batch size and the number of samples collected per rollout, while varying the number of PPO epochs, thereby changing the number of gradient updates per rollout. This improves sample efficiency, with more epochs leading to faster convergence. Results are shown over 1M timesteps. The number of samples collected per rollout was fixed to 2048 for all experiments.

v1, MountainCar-v0, and two continuous action space environments: Pendulum-v1 and MountainCarContinuous-v0. We trained on both environments for 1M steps (1.5M for LunarLander-v2). For NNs we used the hyperparameters provided in RL Baselines3 Zoo (Raffin, 2020), except for AWR where we followed the hyperparameters described in the paper.

Google Research Football & Atari ram. The Football domain (Kurach et al., 2020) consists of a vectorized 115-dimensional observation space that summarizes the main aspects of the game and 19 discrete actions. We focus on its academy scenarios, which present situational tasks involving scoring a single goal. A standard reward of +1 is granted for scoring, and we employed the "Checkpoints" shaped reward structure. This structure provides additional points as the agent moves closer towards the goal, with a maximum reward of 2 per scenario. The Atari-ram domain consists of a vectorized 128-dimensional observational space representing the 128-byte RAM state and up to 18 discrete actions. We trained agents in both domains for 10M timesteps.

MiniGrid. The MiniGrid environment (Chevalier-Boisvert et al., 2023) is a 2D grid world with goal-oriented tasks requiring object interaction. The observation space consists of a 7x7 image representing the grid, a mission string, and the agent's direction. Each tile in the observed image contains a 3D tuple dictating an object's color, type, and state. All MiniGrid tasks emit a reward of +1 for successful completion and 0 otherwise. We trained our NN-based agents on a flattened observation space using the built-in one-hot wrapper, as specified by RL Baselines3 Zoo. For GBRL agents, we generated a 51-dimensional categorical observational space by encoding each unique tile tuple as a categorical string to represent the observed image. Categorical features were added for the agent's direction (up, left, right, down) and missions. All agents were trained for 1M timesteps, except for PutNear, FourRooms, and Fetch tasks, which were trained for 10M based on the reported values for PPO NN in RL Baselines3 Zoo.

Variable Isolation Environment. The goal of the variable isolation task is to separate a target variable within a linear equation. In the basic task variation, the initial state is a linear equation of the form $ax + b = c$, where a, b, c are digits within the range $[1 - 9]$ that can be either positive or negative. The action space is multi-discrete, where the agent chooses an action type, the digit to use, and the sign of the digit. The action types are: addition, subtraction, multiplication, and division. The agent receives a reward proportional to the number of steps taken in the episode if it successfully isolates x, otherwise the reward is 0.

In the two-variable linear equation variation, the initial state is a linear equation of the form $ax + by + c = d$, and the goal is to isolate y on the left side. The actions are equivalent to the basic task, with the addition or subtraction of a digit multiplied by x. Finally, in the balanced two-variable linear equation variation, the initial state is a linear equation of the form $ax + b = cy + d$, and the goal is to isolate y on the left side. The actions are equivalent to the two-variable linear equation variation with the addition of the operations involving y. In all the environment variations, the agent observes the coefficients as inputs. For the two variable variations, we add a small reward bonus if the agent successfully moves a variable or a constant to the correct side. We trained GBRL for 15M steps and NNs for 30M steps.

C.3. Compute Resources

All experiments were done on the NVIDIA NGC platform on a single NVIDIA V100-32GB GPU per experiment. Training time and compute requirements vary between algorithms and according to hyperparameters. The number of boosting iterations has the largest impact on both runtime and memory. GBRL experimental runs required from 1GB to 24GB of GPU memory. Moreover, runtime varied from 20 minutes for 1M timesteps training on classic environments and up to 5 days for 10M timesteps on Atari-ram. NN experimental runs required up to 3GB of GPU memory and runtime ranged from 10 minutes and up to 3 days. The total compute time for all experiments combined was approximately 1800 GPU hours. Additionally, the research project involved preliminary experiments and hyperparameter tuning, which required an estimated additional 168 GPU hours.

D. Detailed Result Tables

This section contains tables presenting the mean and standard deviation of the average episode reward for the final 100 episodes within each experiment. More specifically, Table 3 presents results for Continuous Control & Block2D environments, Tables 4 and 5 present results for the high-dimensional vectorized environments, and Table 6 presents results for the categorical environments.

E. Learning Curves

This section presents learning curves depicting model performance throughout the training phase. Figures 16 to 19 show the training reward as a function of environment steps of the agents trained in the experiments. The column order is: A2C,

	batch size	clip range	ent coef	gae lambda	gamma	num epochs	num steps	num envs	policy lr	value lr
Acrobot	512	0.2	0.0	0.94	0.99	20	128	16	0.16	0.034
CartPole	64	0.2	0.0	0.8	0.98	1	128	8	0.029	0.015
LunarLander	256	0.2	0.0033	0.98	0.999	20	512	16	0.031	0.003
MountainCar	256	0.2	0.033	0.98	0.999	20	512	16	0.031	0.003
MountainCar Continuous	256	0.2	0.033	0.98	0.999	20	512	16	0.031	0.003
Pendulum	512	0.2	0.0	0.93	0.91	20	256	16	0.031	0.013
Football	512	0.2	0.0	0.95	0.998	10	256	16	0.033	0.006
Atari-Ram	64	0.92	8e-5	0.95	0.99	4	512	16	0.05	0.002
MiniGrid	512	0.2	0.0	0.95	0.99	20	256	16	0.17	0.01
Variable Isolation	128	0.62	0.08	0.95	0.99	20	256	16	0.18	0.006

(a) PPO. For continuous action spaces, we used log std init = -2 and log std lr = lin_0.0017. We utilized gradient norm clipping for Gym environments. Specifically, 10 for the value gradients and 150 for the policy gradients.

	ent coef	gae lambda	gamma	num steps	num envs	policy lr	value lr	log std init	log std lr
Acrobot	0.0	1	0.99	8	4	0.79	0.031	-	-
CartPole	0.0	1	0.99	8	16	0.13	0.047	-	-
LunarLander	0.0	1	0.995	5	32	0.16	0.04	-	-
MountainCar	0.0	1	0.99	8	16	0.64	0.032	-	-
MountainCar Continuous	0.0	1	0.995	128	16	0.0008	2.8e-6	0	0.0004
Pendulum	0.0	0.9	0.9	10	32	0.003	0.056	-2	0.00018
Football	0.0004	0.95	0.998	128	8	0.87	0.017	-	-
Atari-Ram	0.0009	0.95	0.993	128	8	0.17	0.013	-	-
MiniGrid	0.0	0.95	0.99	10	128	0.34	0.039	-	-

(b) A2C

	batch size	ent coef	gae lambda	gamma	train freq	gradient steps	num envs	policy lr	value lr	log std init	log std lr
Acrobot	1024	0.0	0.95	0.99	2000	150	1	0.05	0.1	-	-
CartPole	1024	0.0	0.95	0.99	2000	150	1	0.05	0.1	-	-
LunarLander	1024	0.0	0.95	0.99	2000	150	1	0.05	0.1	-	-
MountainCar	64	0.0	0.95	0.99	2000	150	1	0.64	0.032	-	-
MountainCar Continuous	64	0.0	0.95	0.99	2000	150	1	0.089	0.083	-2	lin_0.0017
Pendulum	1024	0.0	0.9	0.9	1000	50	1	0.003	0.07	-2	0.0005
Football	512	0.03	0.95	0.99	750	10	1	0.09	0.00048	-	-
Atari-Ram	1024	0.0	0.95	0.993	2000	50	1	0.0779	0.0048	-	-
MiniGrid	1024	0.0	0.95	0.99	1500	25/100*	1	0.0075	0.005	-	-

(c) AWR. For all envs, buffer size = 50,000, β = 0.05. *MiniGrid environments used 100 gradient steps for tasks trained for 1M steps, and 25 gradient steps for tasks trained for 10M steps, for a reduced tree size.

Table 2. **GBRL hyperparameters** - NN represented by an MLP with two hidden layers.

Table 3. Continuous-Control and Box2D environments: Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

	Acrobot	**CartPole**	**LunarLander**	**MountainCar**	**MountainCar Continuous**	**Pendulum-v1**
NN: A2C	-82.27 ± 3.29	500.00 ± 0.0	-43.01 ± 106.26	-148.90 ± 24.10	92.66 ± 0.32	$\mathbf{-183.64 \pm 22.32}$
GBRL: A2C	-90.73 ± 2.98	500.00 ± 0.0	$\mathbf{47.93 \pm 41.00}$	-124.42 ± 5.74	93.15 ± 1.19	-538.83 ± 66.25
NN: AWR	-102.53 ± 57.25	500.00 ± 0.0	$\mathbf{282.48 \pm 1.96}$	-160.65 ± 53.97	18.93 ± 42.34	$\mathbf{-159.64 \pm 9.42}$
GBRL: AWR	-118.12 ± 33.54	497.54 ± 3.11	76.03 ± 56.62	-146.68 ± 24.53	44.38 ± 45.94	-1257.61 ± 98.10
NN: PPO	-74.83 ± 1.22	500.00 ± 0.0	261.73 ± 6.93	-115.53 ± 1.39	85.81 ± 7.51	-249.31 ± 60.00
GBRL: PPO	-87.82 ± 2.16	500.00 ± 0.0	248.72 ± 59.10	-110.55 ± 15.60	89.42 ± 5.73	-246.89 ± 20.61

AWR, and PPO.

Table 4. Football Academy environments: Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

	3 vs 1 with keeper	Corner	Counterattack Easy	Counterattack Hard	Empty Goal	Empty Goal Close
NN: A2C	1.78 ± 0.10	1.00 ± 0.17	$\mathbf{1.58 \pm 0.35}$	$\mathbf{1.43 \pm 0.17}$	$\mathbf{1.93 \pm 0.05}$	2.0 ± 0.0
GBRL: A2C	1.59 ± 0.17	1.01 ± 0.07	1.11 ± 0.14	1.00 ± 0.05	1.81 ± 0.03	2.00 ± 0.00
NN: AWR	1.50 ± 0.37	1.01 ± 0.04	$\mathbf{1.59 \pm 0.36}$	1.18 ± 0.21	1.90 ± 0.08	1.92 ± 0.17
GBRL: AWR	1.66 ± 0.34	0.92 ± 0.05	0.95 ± 0.05	0.92 ± 0.05	1.93 ± 0.07	2.0 ± 0.0
NN: PPO	1.61 ± 0.05	0.95 ± 0.02	1.43 ± 0.15	1.23 ± 0.18	$\mathbf{1.98 \pm 0.01}$	1.99 ± 0.00
GBRL: PPO	1.63 ± 0.19	1.05 ± 0.20	1.64 ± 0.09	1.23 ± 0.07	1.84 ± 0.06	2.0 ± 0.0

	Pass & Shoot keeper	Run Pass & Shoot keeper	Run to Score	Run to score w/ keeper	Single Goal vs Lazy
NN: A2C	1.41 ± 0.37	1.77 ± 0.08	1.87 ± 0.12	1.25 ± 0.23	$\mathbf{1.65 \pm 0.04}$
GBRL: A2C	1.60 ± 0.21	1.60 ± 0.14	1.82 ± 0.10	1.15 ± 0.08	1.31 ± 0.11
NN: AWR	1.26 ± 0.46	1.15 ± 0.14	1.81 ± 0.14	1.25 ± 0.34	1.28 ± 0.27
GBRL: AWR	1.35 ± 0.37	1.53 ± 0.40	$\mathbf{1.98 \pm 0.01}$	0.99 ± 0.16	1.03 ± 0.12
NN: PPO	1.31 ± 0.13	1.64 ± 0.16	1.91 ± 0.09	1.13 ± 0.06	1.68 ± 0.09
GBRL: PPO	$\mathbf{1.87 \pm 0.09}$	1.85 ± 0.08	1.83 ± 0.04	$\mathbf{1.95 \pm 0.02}$	1.73 ± 0.06

Table 5. Atari-ramNoFrameskip-v4 environments: Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

	Alien	Amidar	Asteroids	Breakout	Gopher
NN: A2C	$\mathbf{1802.24 \pm 323.12}$	$\mathbf{304.62 \pm 55.61}$	$\mathbf{2770.46 \pm 271.97}$	$\mathbf{76.69 \pm 30.08}$	$\mathbf{3533.84 \pm 118.50}$
GBRL: A2C	595.08 ± 43.51	48.71 ± 14.65	1402.66 ± 161.67	11.52 ± 2.34	502.20 ± 341.88
NN: AWR	739.82 ± 303.06	86.32 ± 40.16	$\mathbf{2308.68 \pm 257.72}$	26.57 ± 9.91	1471.93 ± 716.65
GBRL: AWR	829.99 ± 166.48	125.53 ± 25.25	1592.63 ± 109.96	17.32 ± 1.89	913.06 ± 79.95
NN: PPO	$\mathbf{1555.32 \pm 107.59}$	$\mathbf{310.93 \pm 80.13}$	$\mathbf{2309.46 \pm 145.66}$	$\mathbf{32.88 \pm 15.74}$	$\mathbf{2507.84 \pm 108.37}$
GBRL: PPO	1163.86 ± 76.54	186.32 ± 50.63	1514.34 ± 317.46	19.96 ± 1.93	1215.04 ± 81.01

	Kangaroo	Krull	MsPacman	Pong	SpaceInvaders
NN: A2C	$\mathbf{2137.6 \pm 425.64}$	$\mathbf{9325.38 \pm 777.12}$	$\mathbf{2007.64 \pm 116.52}$	$\mathbf{15.39 \pm 4.26}$	$\mathbf{462.30 \pm 35.56}$
GBRL: A2C	948.8 ± 483.80	5291.4 ± 433.35	989.68 ± 100.02	-12.80 ± 11.10	265.36 ± 44.64
NN: AWR	1214.8 ± 313.42	4519.78 ± 522.11	892.31 ± 289.36	-10.25 ± 2.11	842.00 ± 130.51
GBRL: AWR	$\mathbf{1809.26 \pm 37.51}$	$\mathbf{6419.26 \pm 387.76}$	$\mathbf{1641.84 \pm 284.19}$	-11.68 ± 3.79	397.85 ± 566.38
NN: PPO	2487.4 ± 829.65	9167.3 ± 294.30	2069.22 ± 202.48	18.50 ± 1.60	479.77 ± 65.07
GBRL: PPO	2160.8 ± 826.92	6888.66 ± 756.18	2069.22 ± 538.62	15.40 ± 6.55	434.84 ± 31.83

Table 6. MiniGrid environments: Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

	DoorKey-5x5	**Empty-Random-5x5**	**Fetch-5x5-N2**	**FourRooms**	**GoToDoor-5x5**
NN: A2C	0.96 ± 0.00	0.77 ± 0.42	0.43 ± 0.03	0.62 ± 0.19	0.05 ± 0.04
GBRL: A2C	0.96 ± 0.00	0.96 ± 0.00	0.62 ± 0.02	0.51 ± 0.07	0.78 ± 0.02
NN: AWR	0.57 ± 0.52	0.96 ± 0.00	0.90 ± 0.26	0.19 ± 0.12	0.95 ± 0.01
GBRL: AWR	$\mathbf{0.96 \pm 0.00}$	0.97 ± 0.00	$\mathbf{0.95 \pm 0.01}$	$\mathbf{0.54 \pm 0.05}$	0.94 ± 0.01
NN: PPO	0.78 ± 0.40	0.96 ± 0.00	0.89 ± 0.03	0.53 ± 0.03	0.60 ± 0.06
GBRL: PPO	0.96 ± 0.00	0.96 ± 0.00	$\mathbf{0.96 \pm 0.01}$	0.56 ± 0.04	$\mathbf{0.96 \pm 0.00}$

	KeyCorridorS3R1	**PutNear-6x6-N2**	**RedBlueDoors-6x6**	**Unlock**
NN: A2C	0.75 ± 0.42	0.01 ± 0.00	$\mathbf{0.30 \pm 0.22}$	0.77 ± 0.43
GBRL: A2C	0.39 ± 0.48	$\mathbf{0.18 \pm 0.018}$	0.0 ± 0.0	0.90 ± 0.09
NN: AWR	0.93 ± 0.00	$\mathbf{0.60 \pm 0.13}$	0.83 ± 0.00	0.96 ± 0.00
GBRL: AWR	0.94 ± 0.00	0.36 ± 0.01	0.84 ± 0.03	0.95 ± 0.00
NN: PPO	0.76 ± 0.42	0.001 ± 0.00	0.17 ± 0.40	0.97 ± 0.00
GBRL: PPO	$\mathbf{0.95 \pm 0.00}$	$\mathbf{0.44 \pm 0.19}$	$\mathbf{0.88 \pm 0.02}$	0.97 ± 0.00

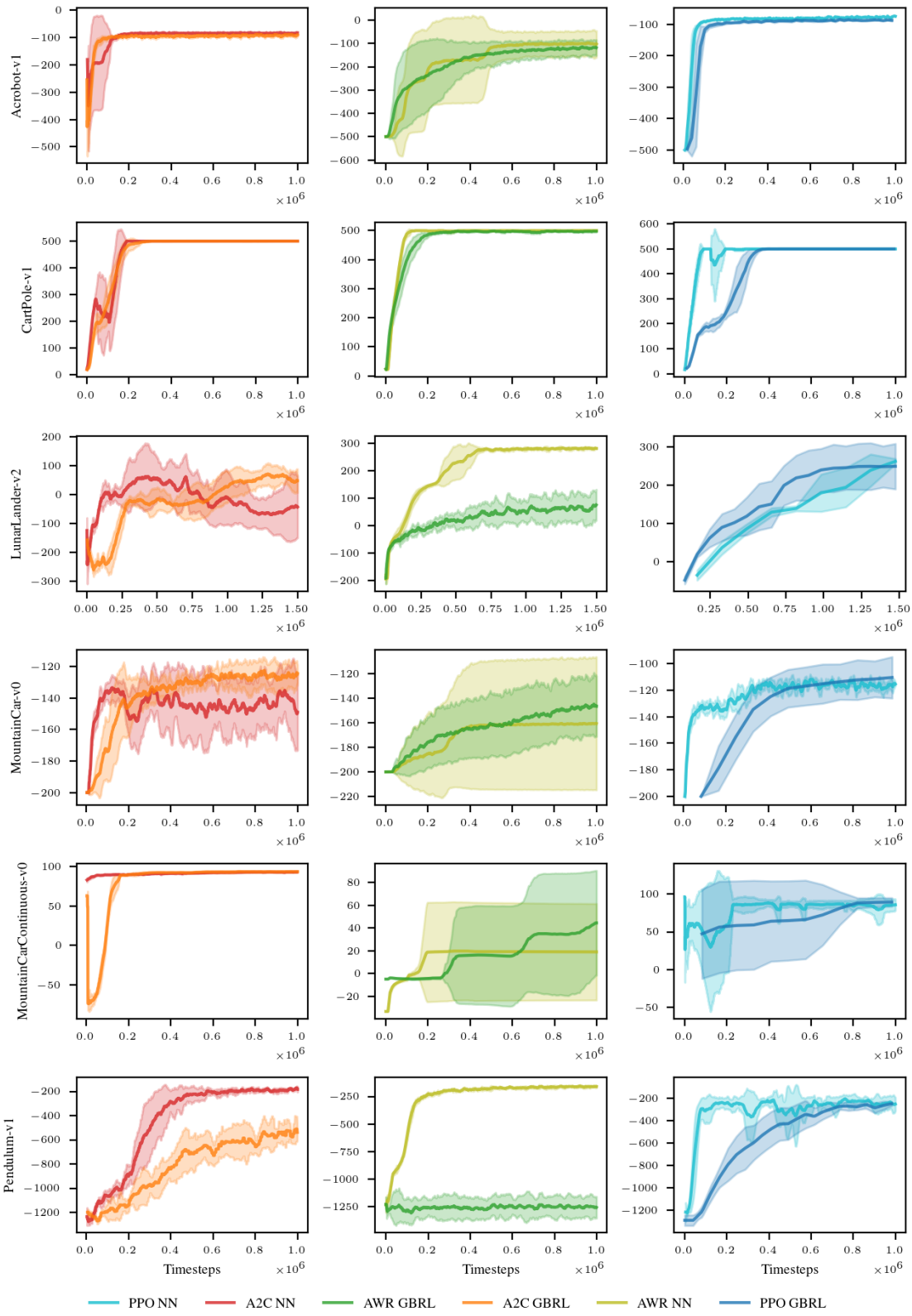


Figure 16. Classic Control and Box2D environments: Training reward as a function of environment steps. Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

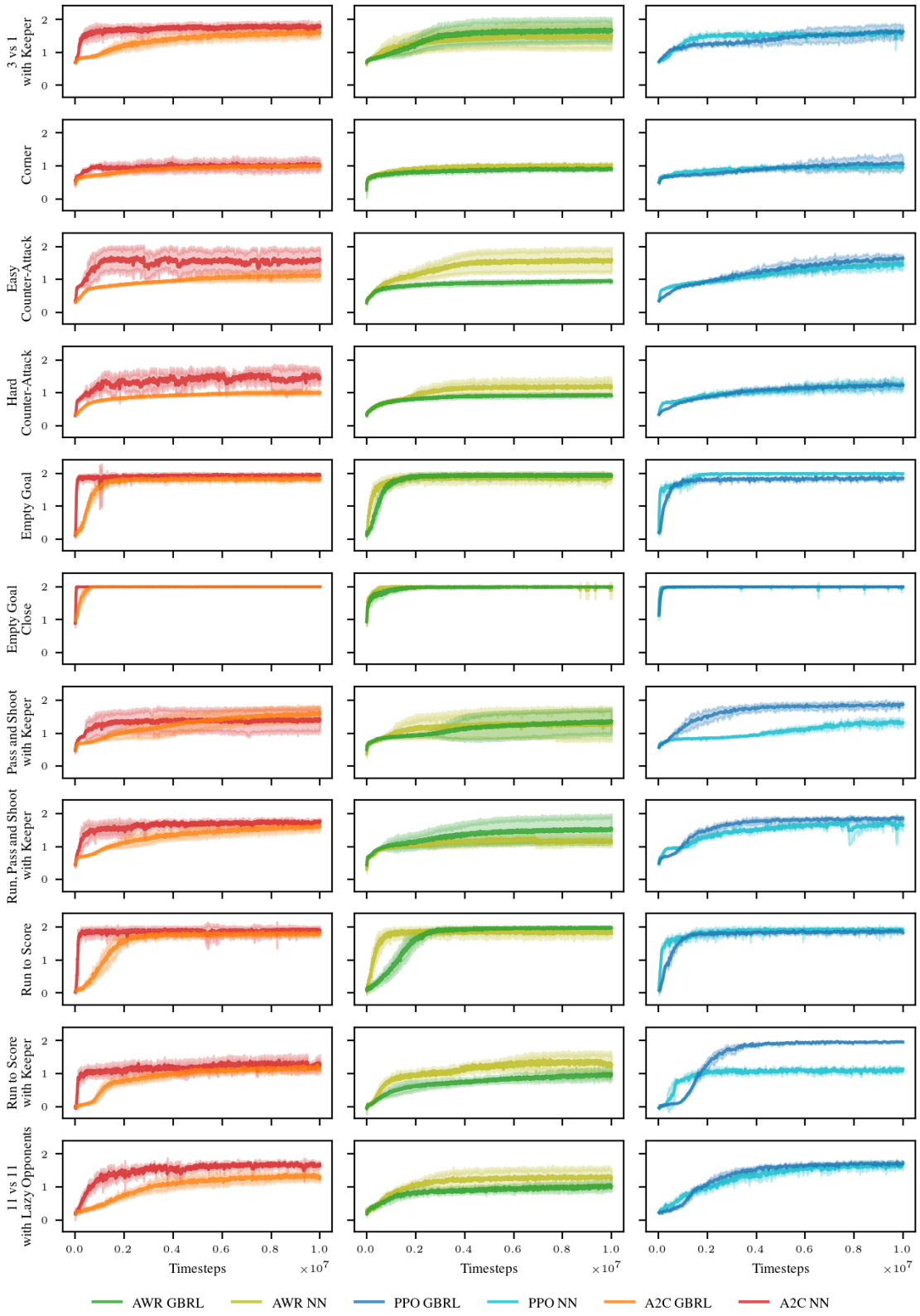


Figure 17. Football Academy environments: Training reward as a function of environment step. Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

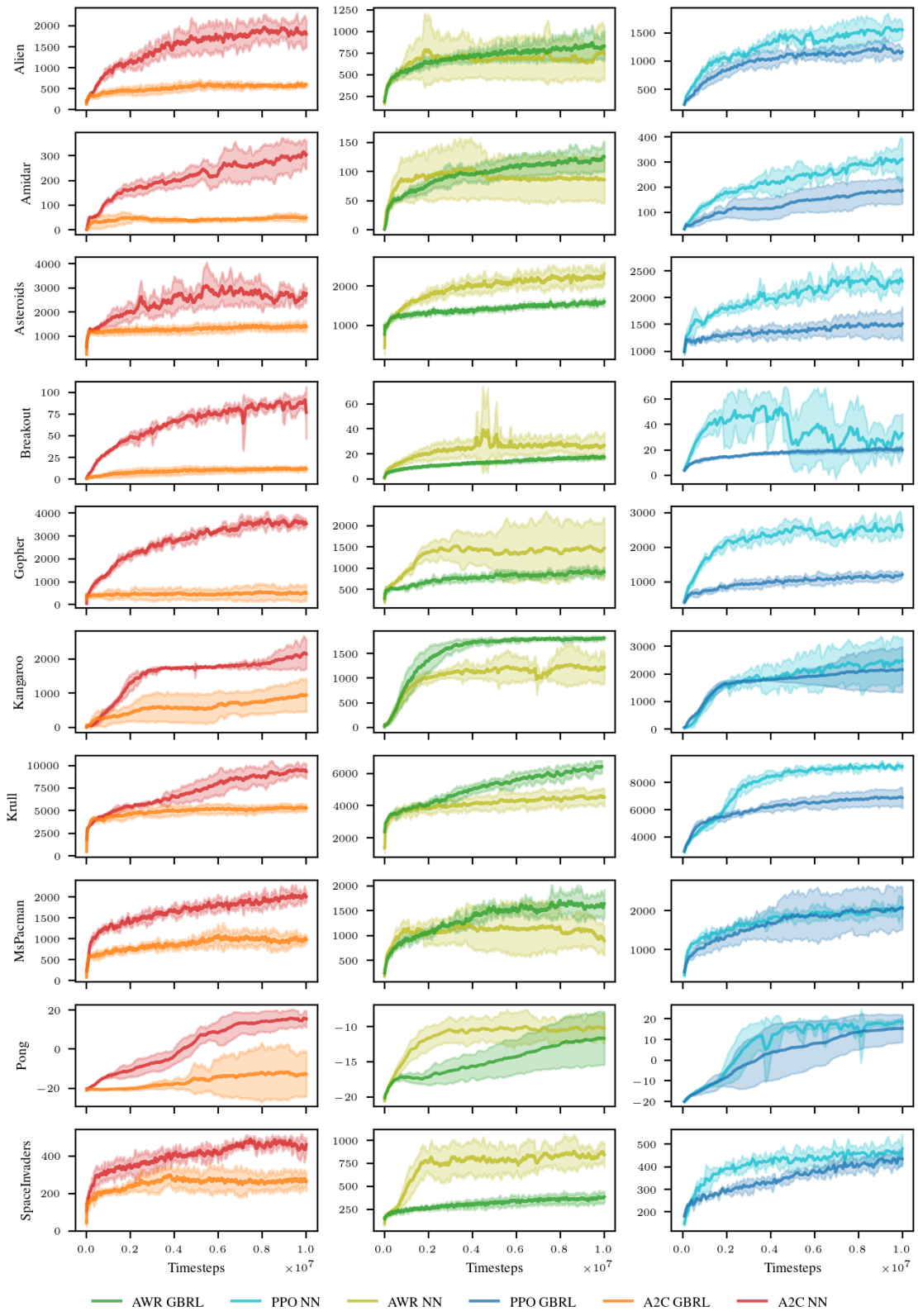


Figure 18. Atari-ramNoFrameskip-v4 environments: Training reward as a function of environment step. Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

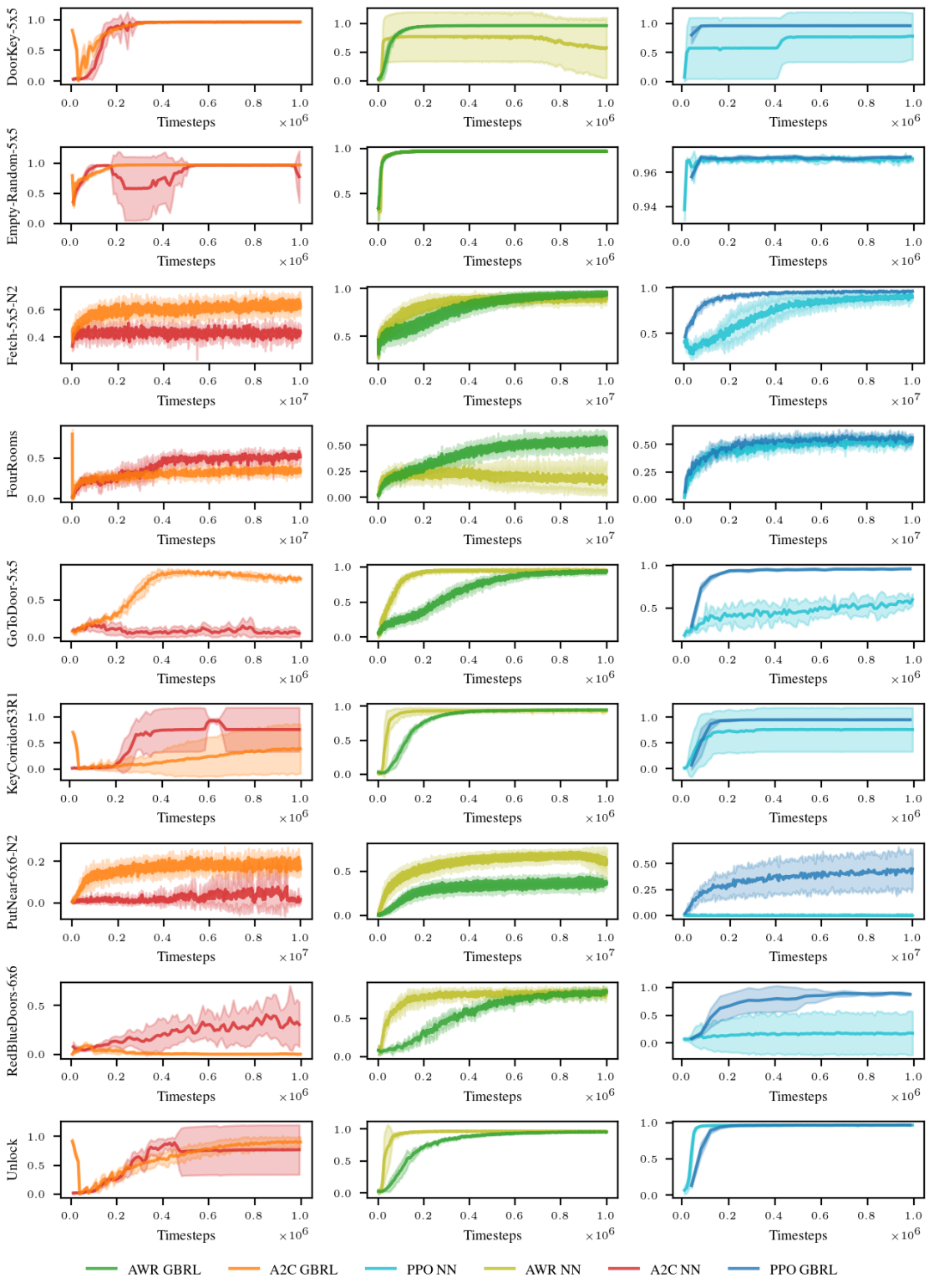


Figure 19. MiniGrid environments: Training reward as a function of environment step. Reported values are the mean and standard deviation of the average episode reward over the final 100 training episodes, aggregated across 5 random seeds.

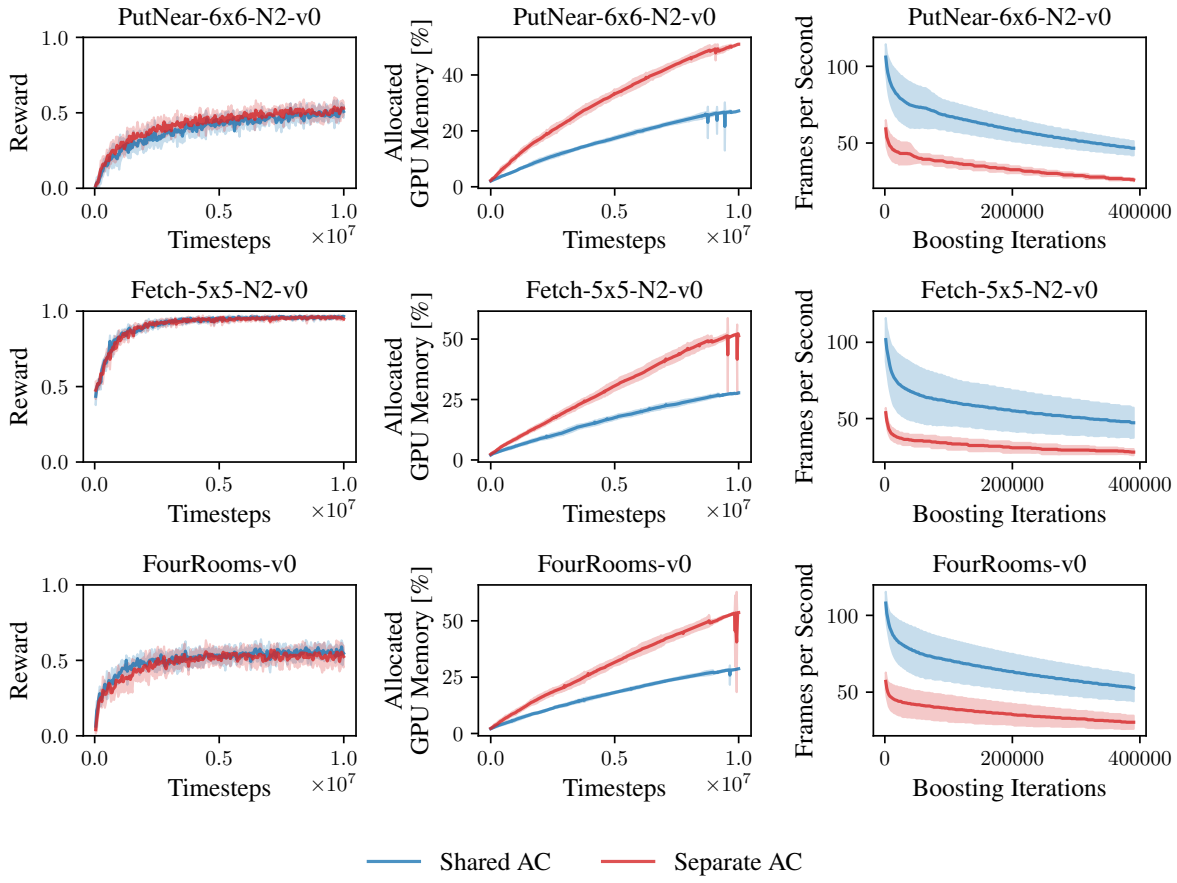


Figure 20. **Sharing actor critic tree structure significantly increases efficiency while retraining similar performance**. Training reward, GPU memory usage, and FPS, are compared across 10M environment (5 seeds, 3 MiniGrid environments)